

Thursday, April 23rd, 2026
Reinforcement Learning Policies for Formula 1 Gym

STAT
4830

Yuv Malik | Matthew Lobo | Pablo Echevarria Cuesta

Problem Statement

GUIDING QUESTIONS

- Can we train an agent to drive a race car autonomously in simulation?
- Can we train multiple agents to drive **competitively** and **safely**?
- Can we extrapolate our simulation to train a world model and get equal or better training results?

1. One-Car Loop

Technical Approach

Architecture: A shared 3-layer CNN encoder feeding into separate MLP heads (256 → 128 units) for the Actor and Critic.

- Policy Network Learns distribution
- Value Network learns the expected return under a policy

Policy Updates:

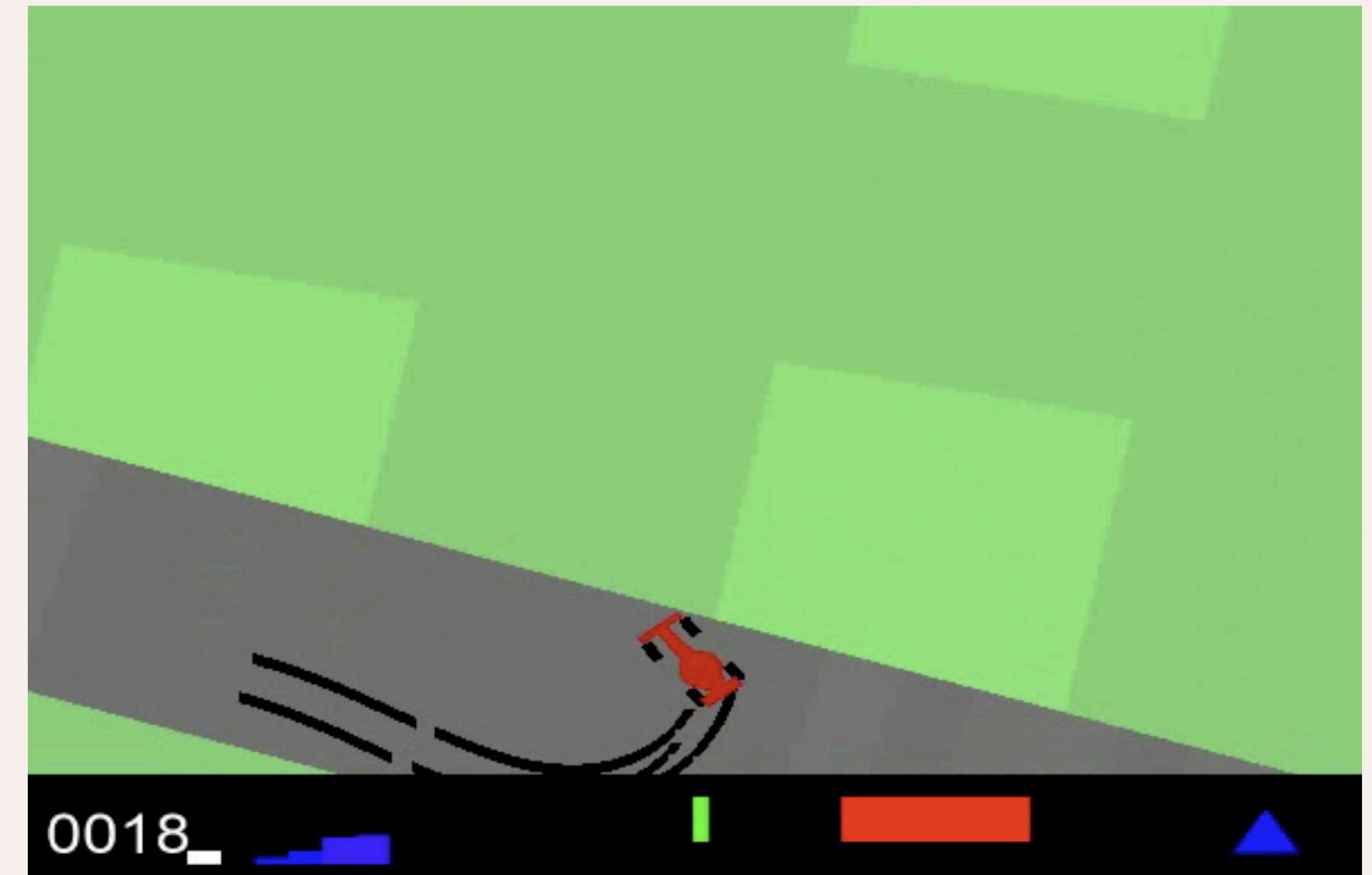
- Using log probability for calculus convenience
- Grad weighted with Advantage (only updates with surprise)
- Calculate advantage comparing current policy vs value head
- Expectation (as we are training in batches)

Issues:

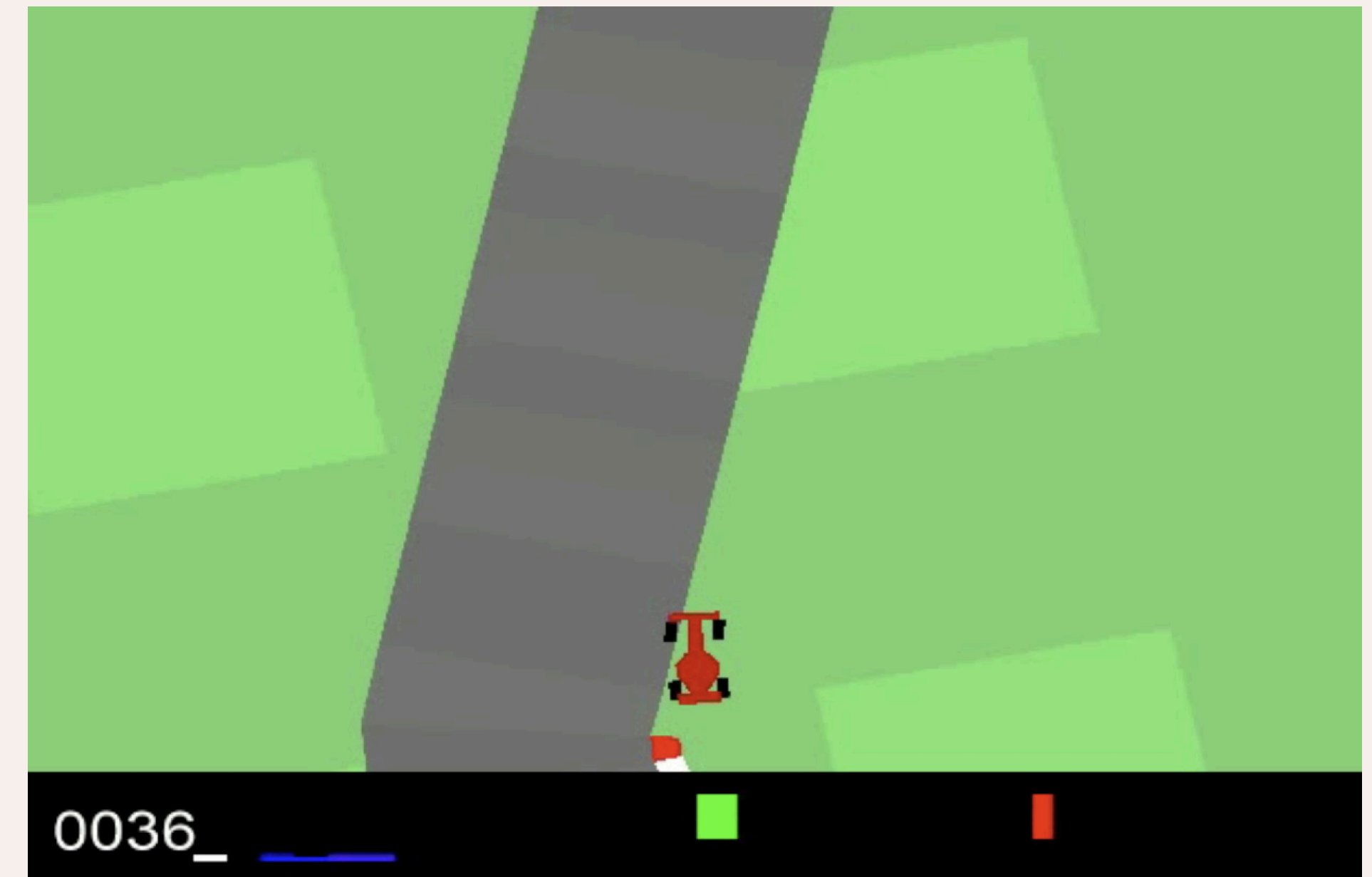
- Lots of car spinning
- Jerky Movement
- Incorrect progress tracking

To address this, we manually added penalties and safeguards to the reward function:

- **Yaw-rate penalty:** punished excessive spinning
- **No-progress termination:** ended episodes when the car stopped making forward progress
- **Progress Reward:** Car rewarded for reaching end goal in fastest time, rather than incrementally.



Rendering showing car getting stuck in loop



Rendering showing car cutting off-track

AutoResearch

When Karpathy tells us something, we listen.

- We used Auto Research to automatically tune PPO after the agent got stuck in failure modes like corner-cutting, spin-outs, and unstable curve handling
- You essentially create a **sandbox** for an LLM to make changes to your research goal. In our case, it had the ability to make code changes to our **train.py** script. It used verifiable rewards from our evaluations to guide its processes.
- The biggest improvement came from changing two key parameters:
 - **Less action noise** → smoother, more stable control
 - **Lower corner target speed** → better turn entry and fewer spin-outs
- These adjustments helped the policy avoid extreme behaviours in curves
- The result was our best single-agent baseline and a much more reliable loop completion policy

2. Two-Car Loop

KEY CHANGES TO SETUP

SHARED-POLICY MARL

Our setup changed from a “one policy controls one car” to a **Shared-Policy Multi-Agent Reinforcement Learning (MARL)**

Our multi-car version now has a SHARED policy that controls all cars.

Instead of every agent having their own “brain”, they all use a **centralized policy** to map their local observations to actions.

BATCHED AGENT STREAMS

Observation & Action tensors now process BATCHED agent streams rather than one stream.

BEFORE:

- Observations: (96, 96, 3)
 - Height (H) - # of pixels rows
 - Width (W) - # of pixel cols
 - Channels - RGB
- Actions (A = 3):
 - Steer | Throttle | Brake

n = # of agents

NOW:

- Observations: (n , 96, 96, 3)
- Actions:
 - (n , A)
- Reward:
 - (n ,)

EXAMPLE: 2 agents

- Observation tensor:
 - (2, 96, 96, 3)
 - index 0 = car1 image
 - index 1 = car2 image
- Policy Outputs Actions
Shape: (2, 3)
 - each row = [steer, throttle, brake] for that car
- Reward Shape: (2,)
-

Those two transitions both go into memory, and PPO updates one shared network from both.

NEED To Change Reward Function !!

Before even attempting to train the cars to race each other, there are some obvious things that needed to be **changed** in the reward function. These were the things we deemed to be the most important:

Ranking

- ranking_reward
- relative_velocity
- overtake_bonus

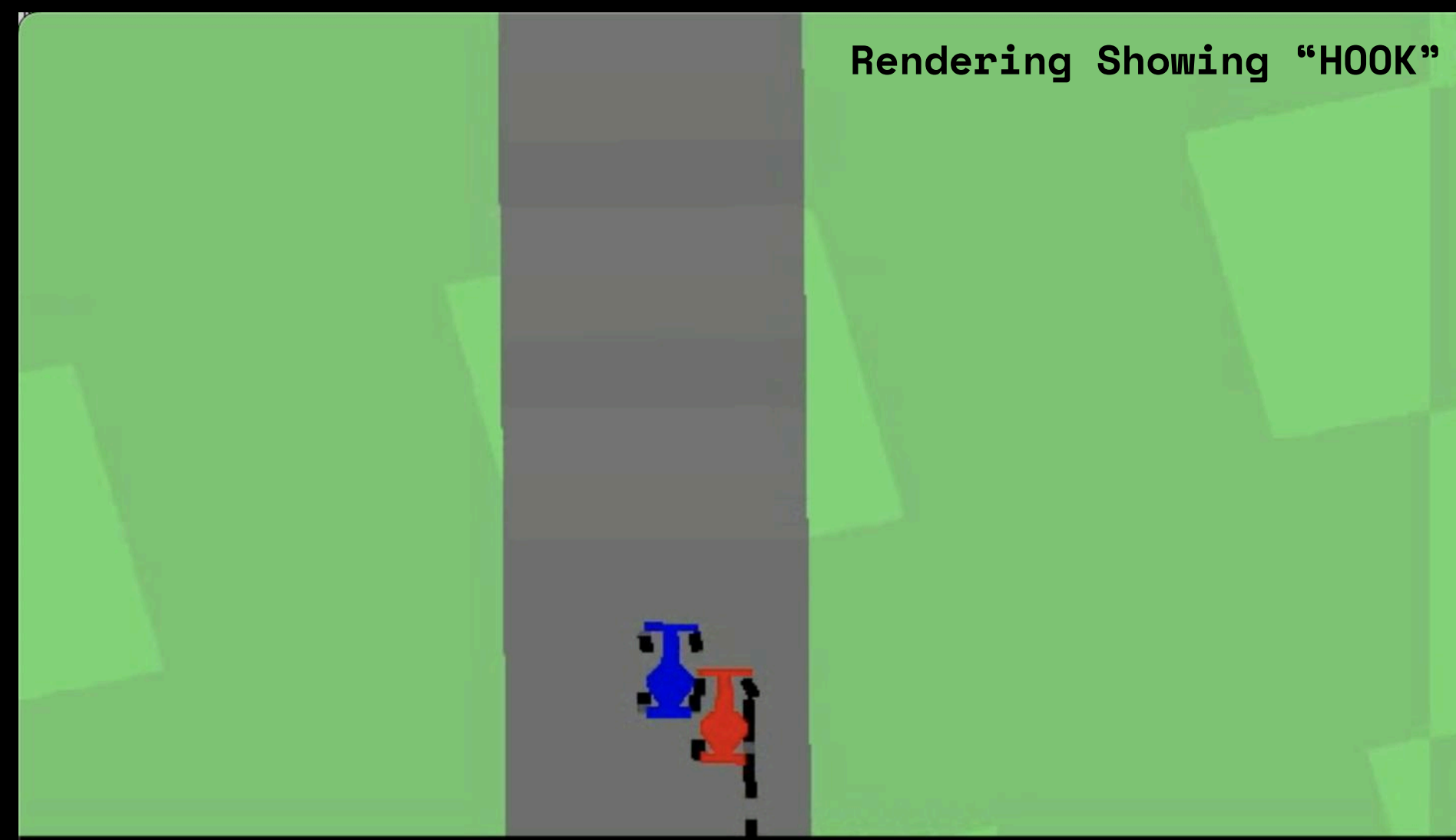
Collisions

- front_gap
- side_clearance
- proximity_penalty

Single-Agent Basics

- Don't change what already works!

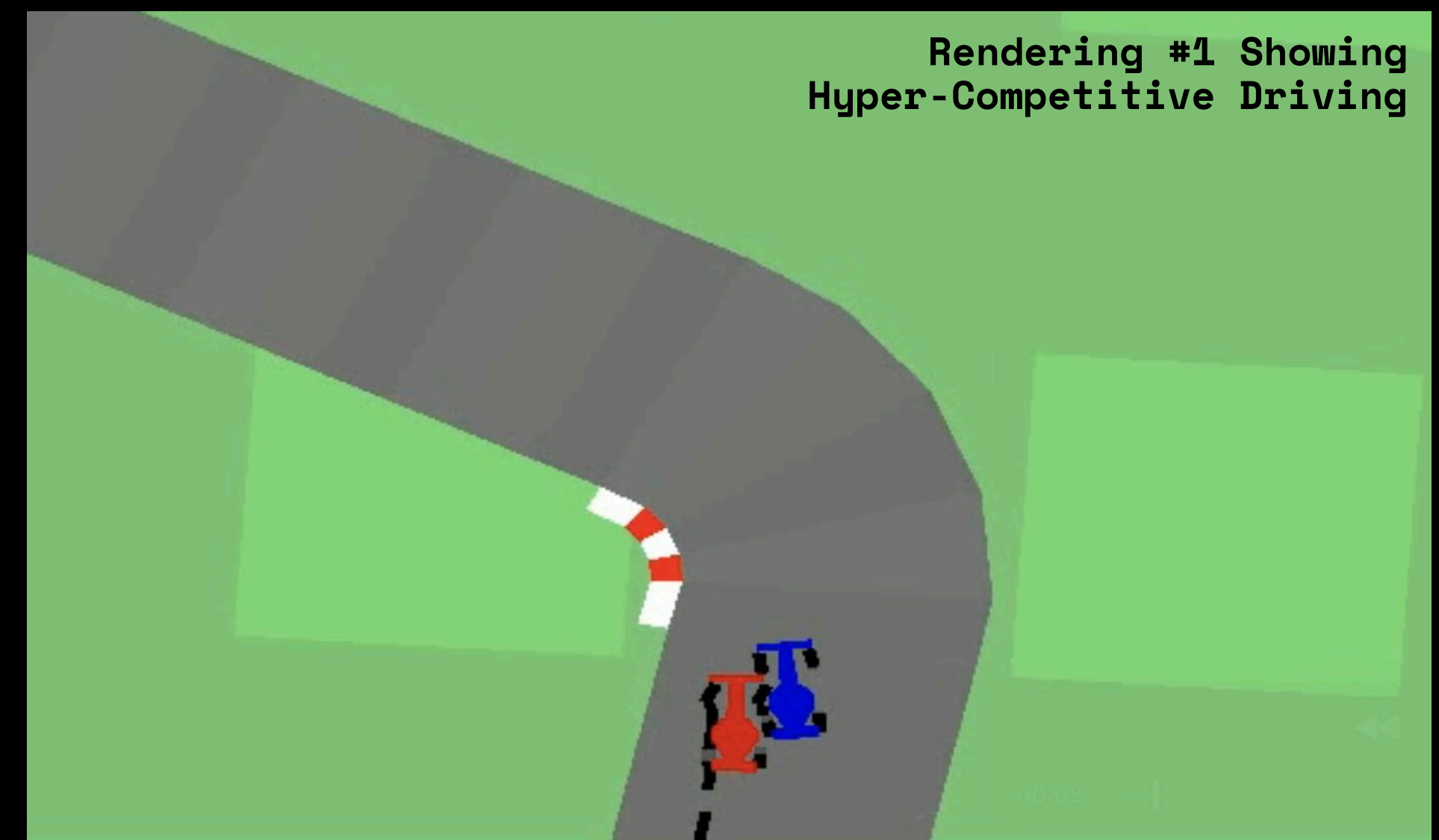
Unrealistic Racing Results



0084



0260



0016



0070

Changing the reward function AGAIN

Seeing these results, we realized that we had not even thought of certain issues that could emerge. Thus, we changed the following:

1. Shift in Mentality

- Competition First → Control/Safety First

2. Strong single-car control

- Reduce speed going into turns from 8.0 to 7.0
- Increase steering smoothness from 0.02 to 0.03

3. Make collision/contact penalties more HARSH

- Penalize:
 - “Hook” contact
 - Sustained contact
 - Collisions:
 - Low | Medium | High (based on speed)
- Terminate when there is prolonged contact
- Spacing & Proximity terms introduced:
 - front_gap ; side_clearance
 - Clean Overtakes

Realistic Racing Results!



10-Agent Racing!!

Now that we successfully had
2 cars racing competitively,
we thought it would be fun to
simulate an even more
“realistic” race - with **10 cars!**



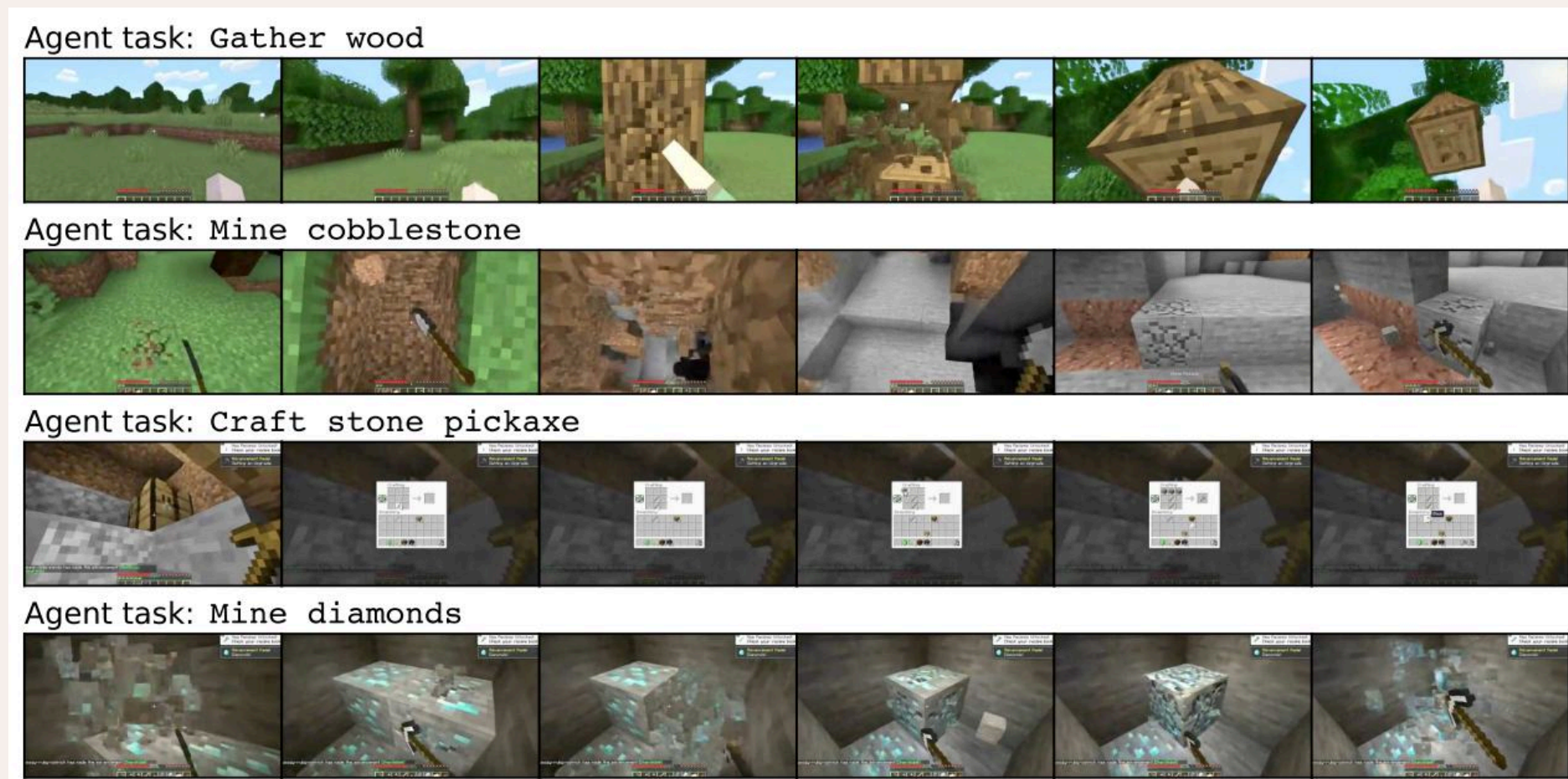
Building a
world model

What is a world model (WM).

Rather than predicting the next word (like LLM's), WM's try to predict the next moment in reality. It learns the 'rules of the game'. How physics works, where the walls are, and what happens when you turn the steering wheel (through some guided trial and error)

The model learns to “dream” what the next best state might be. The edge comes from utilising actor-critic training in this “dream” state (which is nothing but a neural net).

This allows us to train racing strategies thousands of times faster than real-time, bypassing the hardware bottlenecks that usually slow down AI development.



Dreamer V4 - Google Deepmind

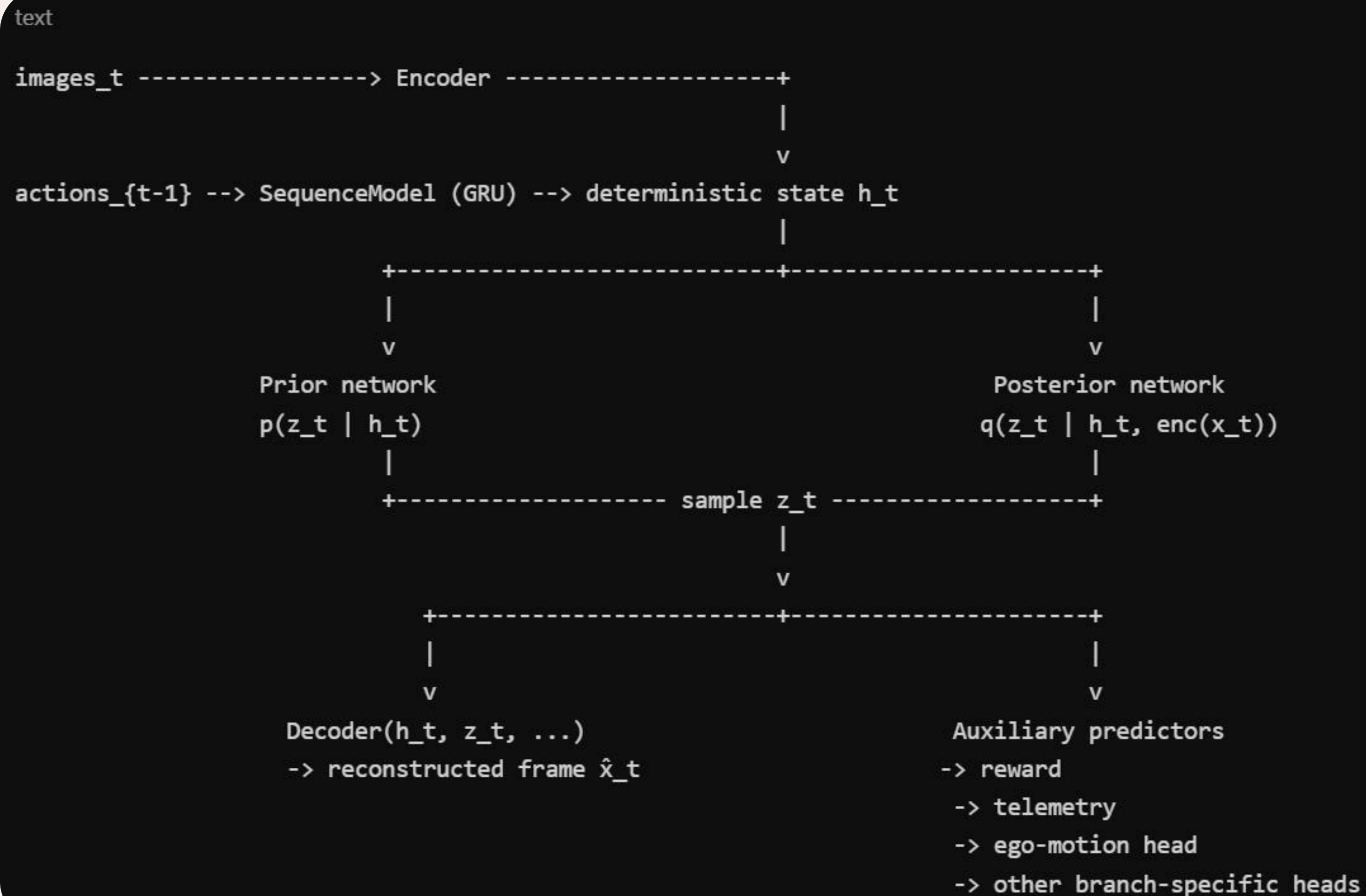
GUIDING QUESTIONS

- (initial) Can we achieve equal or better car training by developing a world model to train in, rather than sim?
- (revised for time) Can we generate 150 frames of geometrically consistent hallucination?

WHY THE WORLD MODEL

- Differentiable. PPO is not.
- No longer have to run env steps in a full physics simulator
- Latent rollouts are far more efficient (energy and compute wise)
- Offline policy rather than online policy.

- Recurrent Neural Network (keeps the “memory” of the past actions/states)
- The RNN predicts a latent state based on past actions.
- Prior network is what our “dreamer” samples (does not get input image)
- Posterior network is what is trainer (does get an input image - use KL divergence to pull prior closer)
- Reconstruction loss from “predicted” latent state



1. Transition core (`SequenceModel` GRU) updates deterministic state

$$h_t = f(h_{t-1}, z_{t-1}, a_{t-1})$$

2. Dynamics prior predicts what latent should be next **without looking at the frame**

$$p(z_t | h_t)$$

3. Posterior infers latent from real frame

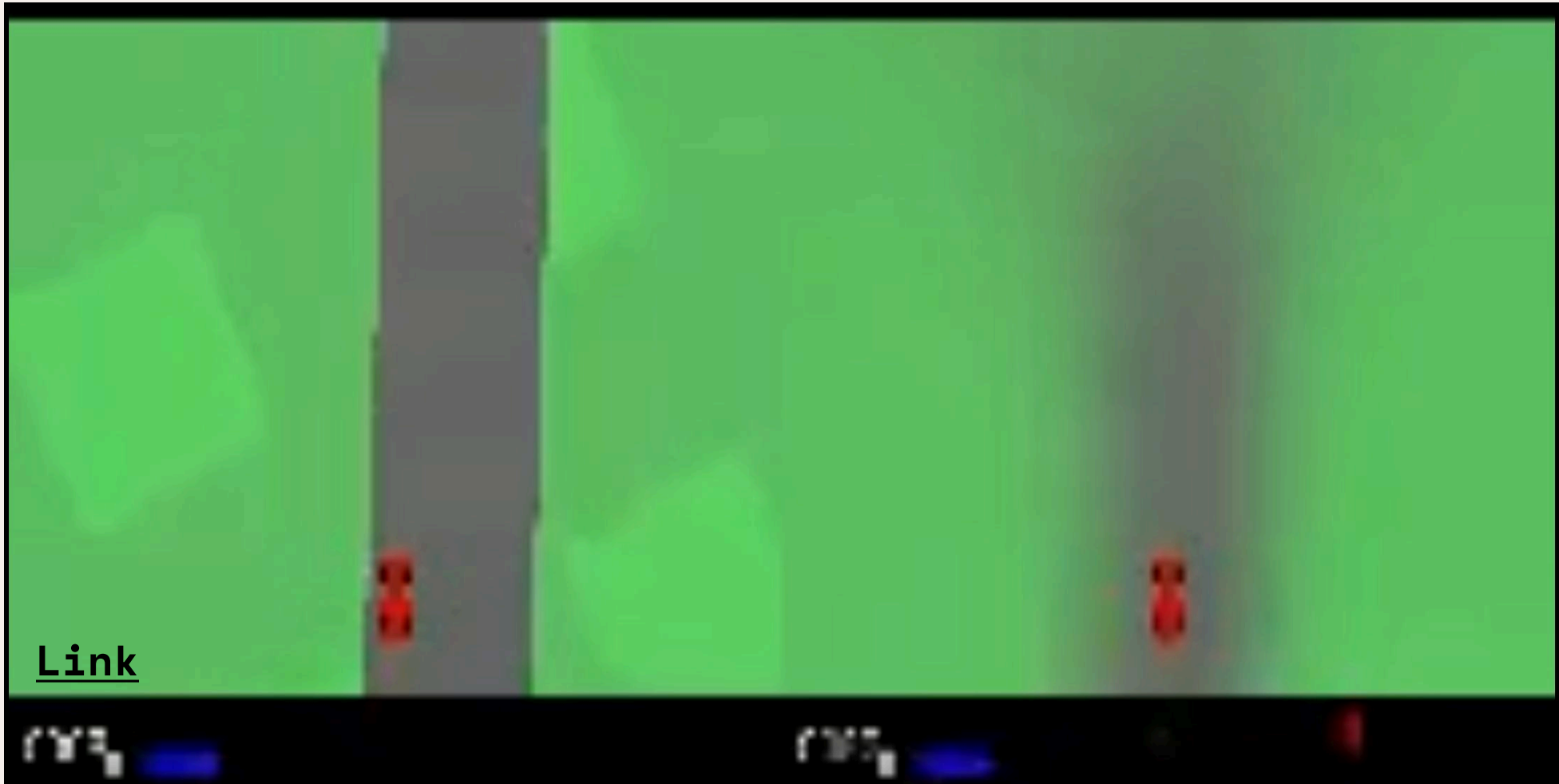
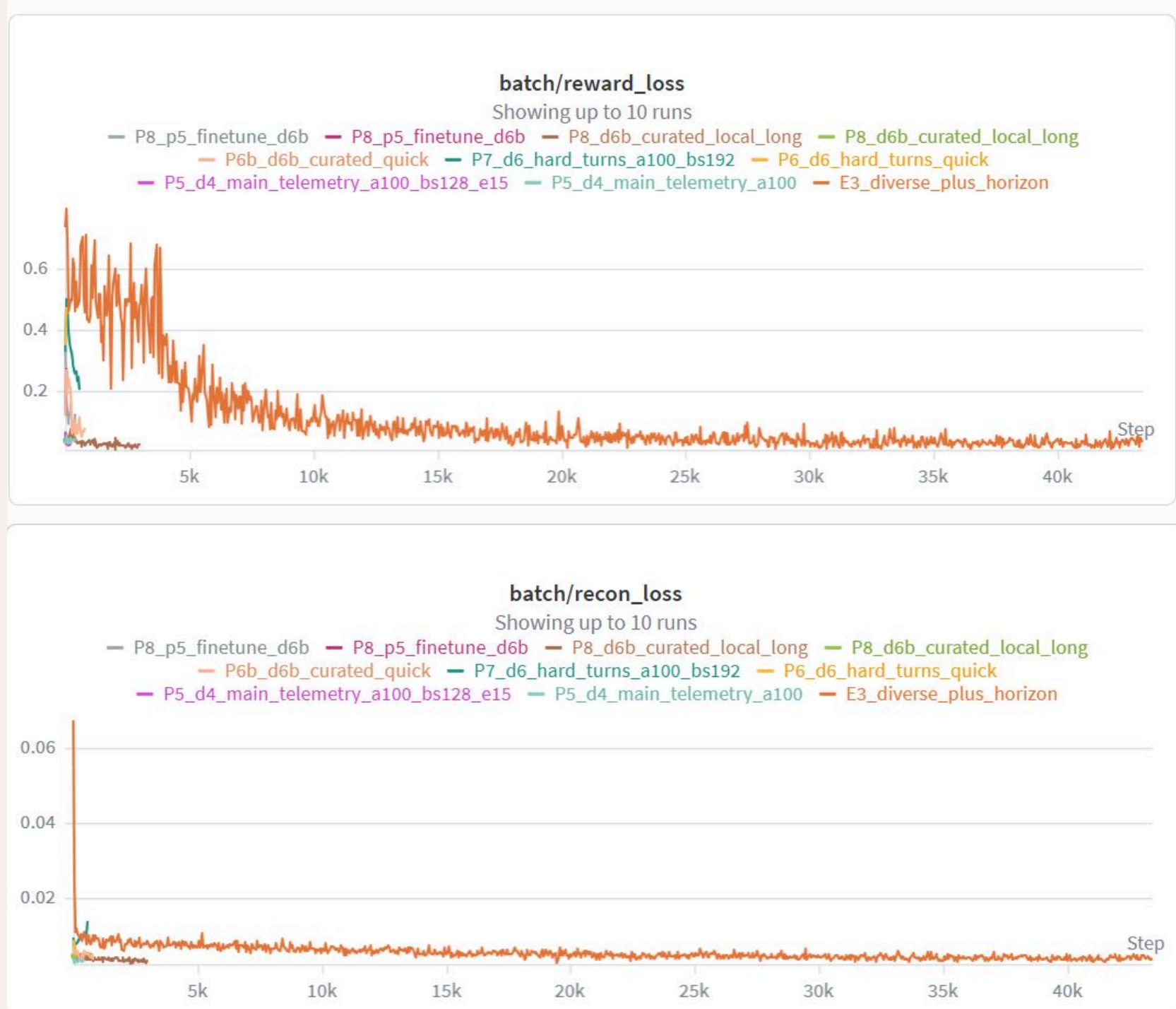
$$q(z_t | h_t, x_t)$$

4. KL term forces prior and posterior to agree:

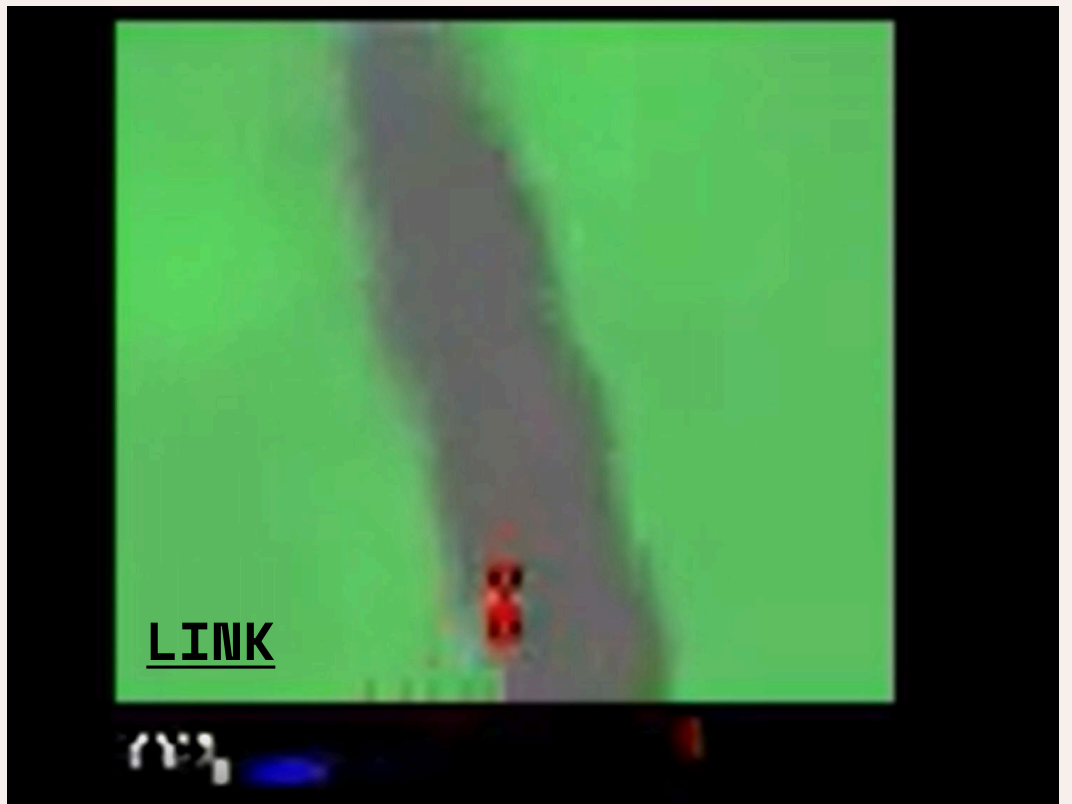
$$L_{KL} = \text{KL}(q(z_t | h_t, x_t) \parallel p(z_t | h_t))$$

- H_t (temporal history)
- Z_t (Latent state/curr state)
- E_t (encoded CNN image)

Our biggest problem was the **nuanced/error carry** forward nature of the latent state.



On the right, we display our strongest (its not very strong) hallucination. Essentially, we feed the first 50 frames and the world model is responsible for displaying the next 50 frames, switching from sampling the posterior to sampling the prior.



FINETUNING METHODS

- Found a lot of problems with the camera (track moved around the car rather than car moving around the track)
 - Added speed/steering/offtrack/ego following to the loss function, so the car could not reward hack the physics.
 - The main failure mode was not generic reconstruction, but poor ego-motion and camera evolution through turns. (Sim Problem)
- Training Data - Could not just train on perfect PPO data like previously
 - Essentially like teaching a baby how the physics of the world works
 - Did a lot of manual data gathering with all sorts of driving (on grass, slow, spinning etc)
 - **Conclusion:** too much noisy data degraded simulator stability and visual consistency even with better behavioural coverage.
- Changing latent state dimensionality
 - Increased latent space from 32 - D to 64 - D after considerable geometric losses in sharp turns etc.
 - **Conclusion:** Larger latent state doesn't necessarily mean more information will be parsed. Model capacity not equal to better factorization.